



Multi-angle Constant Multiplier Givens Rotation Algorithm

Dušan N. Grujić¹ · Lazar Saranovac¹

Received: 1 January 2018 / Revised: 7 February 2019 / Accepted: 9 February 2019
© Springer Science+Business Media, LLC, part of Springer Nature 2019

Abstract

This paper presents a new algorithm for implementing exact Givens rotation for use in QR matrix decomposition. The algorithm is based on constant multipliers to perform multiple angle rotations in parallel, reducing latency and gate count, and is called multi-angle constant multiplier. Complexity and optimal angle division sequences have been studied for up to 20 bits of precision. Bit-exact software implementation of the algorithm was used in QR decomposition of 4×4 matrices with 16 bits of precision, and its numerical stability was extensively tested. QR decomposition was implemented in Xilinx Virtex 6, where pipelining strategies for latency-operating frequency trade-offs have been considered.

Keywords Givens rotation · Constant multiplier · Matrix decomposition · QR

1 Introduction

Wireless data communications capacity is steadily growing, and the use of complex modulations, MIMO and advanced digital signal processing is common. High data rates require fast hardware matrix inversion of real 4×4 real matrices, which is usually performed by QR decomposition [2,8–10,15,16]. Low-latency and low-complexity QR decomposition is one of the key enablers for high-data-rate wireless systems and has been a topic of extensive research in recent years.

In this paper, we present a new algorithm for implementing exact Givens rotation used in QR decomposition. The algorithm uses constant multipliers for implementing rotations by multiple angles in parallel, hence the name multi-angle constant multiplier (MACM). The core idea of the MACM algorithm relies on efficient calculation of multi-angle rotations by using multiplierless multiple constant multiplication

✉ Dušan N. Grujić
dusan.grujic@etf.bg.ac.rs

Lazar Saranovac
laza@etf.bg.ac.rs

¹ School of Electrical Engineering, Bul. kralja Aleksandra 73, Belgrade 11000, Serbia

(MMCM). MMCM is extensively studied in the literature [3,4,14], and software for generating MMCMs from given constants is freely available [13]. The novelty of the proposed algorithm is the choice of rotation angles for elementary Givens rotations which result in MMCM with minimum number of adders.

The proposed algorithm implements the Givens rotation, so it can be used as a drop-in replacement of CORDIC, but with several important differences. CORDIC can be implemented in two ways—with binary angle decision at each stage, when the results are scaled by a constant, and with ternary angle decision, when the results are scaled by a variable number depending on the angle choices [7,11]. In most cases the binary angle decision variant is chosen for simplicity, but it introduces unnecessary rotations and increased round-off errors. Ternary angle decision variant does not suffer from increased round-off error, but requires additional circuitry to calculate the scaling coefficient and a multiplier. MACM algorithm does not introduce the scaling factor, since it implements the Givens rotation exactly, and avoids the accumulation of round-off errors by skipping the unnecessary rotations.

MACM algorithm and detailed comparison to CORDIC are presented in Sect. 2. Bit-exact software implementation of MACM algorithm used for 4×4 matrix QR decomposition, with the study of numerical properties, is given in Sect. 3, while Sect. 4 presents the hardware implementation in FPGA. Final remarks and conclusion are given in Sect. 5.

2 MACM Algorithm

Real matrix $\mathbf{A}$ can be decomposed into a product $\mathbf{A} = \mathbf{QR}$ of orthogonal matrix $\mathbf{Q}$ and upper triangular matrix $\mathbf{R}$ by QR decomposition [6, pp. 223]. One algorithm for QR decomposition uses the Givens rotation matrix $\mathbf{G}(i, j, \theta)$ [6, pp. 215]. Multiplying the Givens rotation matrix from the left with matrix $\mathbf{A}$ affects only rows i and j and is equivalent to counterclockwise rotation of vectors $(a_{i,k}, a_{j,k})$, $k = 1..n$, formed from elements of rows i and j by angle θ . Elements of resulting matrix $\mathbf{A}' = \mathbf{G}(i, j, \theta)\mathbf{A}$ are equal to $\mathbf{A}$, except in rows i and j :

$$\begin{bmatrix} a'_{i,1} & \cdots & a'_{i,n} \\ a'_{j,1} & \cdots & a'_{j,n} \end{bmatrix} = \mathbf{ROT}(\theta) \begin{bmatrix} a_{i,1} & \cdots & a_{i,n} \\ a_{j,1} & \cdots & a_{j,n} \end{bmatrix}. \quad (1)$$

The rotation matrix $\mathbf{ROT}(\theta)$ is given by:

$$\mathbf{ROT}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (2)$$

The choice of rotation angle

$$\theta = -\arctan(a_{j,k}/a_{i,k}) \quad (3)$$

results in $a'_{j,k} = 0$, zeroing one element of matrix $\mathbf{A}$.

Upper triangular matrix $\mathbf{R}$ can be calculated by successively zeroing the elements below the main diagonal:

$$\mathbf{R} = \mathbf{G}(i_1, j_1, \theta_1) \cdots \mathbf{G}(i_n, j_n, \theta_n) \mathbf{A} \quad (4)$$

Having in mind that $\mathbf{Q}$ is orthogonal, i.e., $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$, and multiplying $\mathbf{Q}^T$ from the left with $\mathbf{A} = \mathbf{QR}$, results in $\mathbf{Q}^T \mathbf{A} = \mathbf{R}$, from which it follows:

$$\mathbf{Q}^T = \mathbf{G}(i_1, j_1, \theta_1) \cdots \mathbf{G}(i_n, j_n, \theta_n) \mathbf{I} \quad (5)$$

Therefore, both $\mathbf{R}$ and $\mathbf{Q}$ are determined by the sequence of Givens rotations which triangularize the matrix $\mathbf{A}$.

The rotation by angle θ can be performed as a sequence of micro-rotations by angles θ_i , where $\theta = \sum_i \theta_i$, since $\mathbf{ROT}(\theta_1 + \theta_2) = \mathbf{ROT}(\theta_1) \mathbf{ROT}(\theta_2)$:

$$\mathbf{ROT}(\theta) = \prod_{\theta_i} \mathbf{ROT}(\theta_i), \quad (6)$$

The order of micro-rotations is not important because the rotation matrix (2) is commutative. Micro-rotations are used by the CORDIC [5] implementation of Givens rotation, where the i th angle is:

$$\theta_{i,\text{CORDIC}} = \arctan 2^{-i}, \quad (7)$$

resulting in rotation matrix:

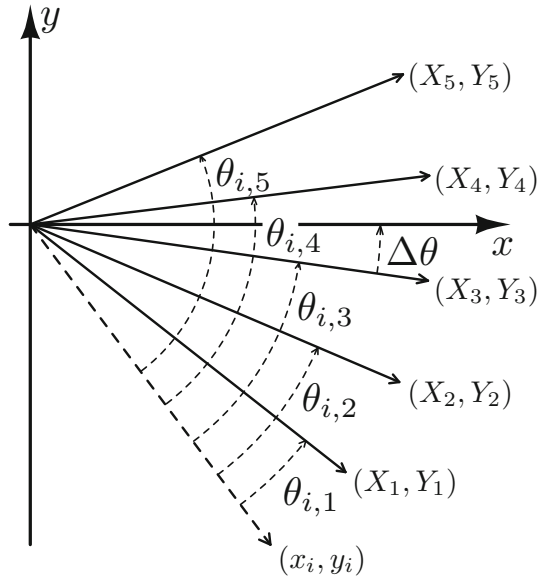
$$\mathbf{ROT}_{\text{CORDIC}}(\theta_i) = \left(1 + 2^{-2i}\right)^{-0.5} \begin{bmatrix} 1 & -\sigma_i 2^{-i} \\ \sigma_i 2^{-i} & 1 \end{bmatrix}, \quad (8)$$

where σ_i can be $\{-1, 0, 1\}$. Total rotation angle $\theta = \sum_i \sigma_i \theta_i$ is uniquely determined by σ_i . The rotation matrix (8) has a desirable property that its elements are either one or powers of two, which can be implemented with shift operations, but also contain a scaling factor $(1 + 2^{-2i})^{-0.5}$. If σ_i is restricted to $\{-1, 1\}$, which corresponds to performing the rotation at each step, even if it is not necessary, the overall scaling factor is constant and can be compensated with a constant multiplier.

Zeroing of each element consists of two steps: determining the rotation angle θ (3) and rotating the elements of rows i and j (1). In the vectoring mode of operation, the CORDIC algorithm sets σ_i so that the vector is rotated toward the positive x axis. By doing so, it zeroes the y component of the vector and determines the angle θ (3). Remaining elements of rows i and j are rotated with values of σ_i determined in the vectoring step. Since the angle of rotation is determined and used bit by bit, the rotations can commence right after the first bit of angle is determined, allowing efficient pipelining.

The main idea of our algorithm is to perform micro-rotations with unity scaling factor by implementing (2) exactly, so unnecessary rotations may be skipped, and to

Fig. 1 Input vector (x_i, y_i) rotated by five angles $\theta_{i,1} - \theta_{i,5}$ in parallel. Vector $(x_{i+1}, y_{i+1}) = (X_3, Y_3)$ is passed to output, with residual angle $\Delta\theta$



decrease the number of iterations by resolving multiple angles in parallel. Skipping unnecessary rotations and reducing the number of iterations will result in lower latency and rounding errors.

As in the CORDIC algorithm, our algorithm determines the rotation angle θ iteratively by rotating the vector toward the positive x axis. Assuming that the input vector is always in the fourth quadrant, the maximum angle of rotation is $\pi/2$. In i th iteration, the vector (x_i, y_i) is rotated by m_i angles $\{\theta_{i,1}, \dots, \theta_{i,m_i}\}$ in parallel. Multi-angle micro-rotation can be expressed as:

$$\begin{bmatrix} X_j \\ Y_j \end{bmatrix} = \mathbf{ROT}(\theta_{i,j}) \begin{bmatrix} x_i \\ y_i \end{bmatrix}, \quad j = 1..m_i. \quad (9)$$

Illustration of multi-angle micro-rotation is shown in Fig. 1 for the case $m_i = 5$. Vector (x_i, y_i) , shown with dashed line, is rotated by five angles $\theta_{i,1}, \dots, \theta_{i,5}$. The goal is to align the vector with the positive x axis, so the vector with smallest residual angle $\Delta\theta$ should be selected. Since the residual angle is not known, the vector in fourth quadrant closest to the x axis is selected for output. This also results in simple decision circuit, as will be shown shortly. For the case shown in Fig. 1, vector (X_3, Y_3) is selected as the output. If all of the rotated vectors are in the first quadrant, the rotation is skipped and the input vector (x_i, y_i) is passed to the output. Rotation angle for i th iteration is selected as:

$$\sigma_i = \max [\{0\} \cup \{k | Y_k < 0\}], \quad (10)$$

and should be saved for subsequent rotations. In two's complement representation, sign bits of $(Y_{m_i}, \dots, Y_i)$ represent σ_i in thermometer code. As in the case of CORDIC

algorithm. the rotation angle is both determined and used iteratively, allowing efficient pipelining. Output selection is then:

$$\begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} = \begin{cases} [x_i \ y_i]^T & , \sigma_i = 0 \\ [X_{\sigma_i} \ Y_{\sigma_i}]^T & , \sigma_i > 0 \end{cases} \quad (11)$$

Algorithm for MACM implementation of Givens rotation is summarized in Algorithm 1. Running the algorithm in vectoring mode determines the angle of rotation (3) which zeroes out one element of the matrix. Givens rotation is completed by running the algorithm in rotation mode on remaining elements of selected rows.

Algorithm 1 MACM implementation of Givens rotation

Require: Input vector $[x_{in}, y_{in}]$, rotation angles $\theta_{i,j}$, $i = 1..k$, $j = 1..m_i$

```

if vectoring then
  {Ensure that the vector is in the fourth quadrant}
  if  $x_{in} > 0 \wedge y_{in} > 0$  then
     $r \leftarrow 3$ 
  else if  $x_{in} > 0 \wedge y_{in} < 0$  then
     $r \leftarrow 0$ 
  else if  $x_{in} < 0 \wedge y_{in} > 0$  then
     $r \leftarrow 2$ 
  else
     $r \leftarrow 1$ 
  end if
end if
{Apply the pre-rotation determined in vectoring mode}
if  $r = 0$  then
   $x_0 \leftarrow x_{in}, y_0 \leftarrow y_{in}$ 
else if  $r = 1$  then
   $x_0 \leftarrow -y_{in}, y_0 \leftarrow x_{in}$ 
else if  $r = 2$  then
   $x_0 \leftarrow -x_{in}, y_0 \leftarrow -y_{in}$ 
else
   $x_0 \leftarrow y_{in}, y_0 \leftarrow -x_{in}$ 
end if
for  $i = 1..k$  do
   $\begin{bmatrix} X_j \\ Y_j \end{bmatrix} \leftarrow \text{ROT}(\theta_{i,j}) \begin{bmatrix} x_i \\ y_i \end{bmatrix}, j = 1..m_i$  {Perform  $m_i$  rotations in parallel}
  if vectoring then
     $\sigma_i \leftarrow \max [ \{0\} \cup \{k \mid Y_k < 0\} ]$  {Select vector in fourth quadrant closest to  $x$  axis}
  end if
   $\begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} \leftarrow \begin{cases} [x_i \ y_i]^T & , \sigma_i = 0 \\ [X_{\sigma_i} \ Y_{\sigma_i}]^T & , \sigma_i > 0 \end{cases}$  {Output the selected vector}
end for
return  $(x_{k+1}, y_{k+1})$ 
  
```

In principle, the choice of rotation angles $\theta_{i,j}$ is constrained only by condition $0 < \theta_{i,1} < \dots < \theta_{i,m_i} < \varphi_i$, where φ_i is the maximum angle which needs to be resolved in i th iteration and $\varphi_1 = \pi/2$. The next iteration must be able to resolve the maximum residual angle, so:

$$\varphi_{i+1} \geq \max(\theta_{i,1}, \theta_{i,2} - \theta_{i,1}, \dots, \varphi_i - \theta_{i,m_i}). \quad (12)$$

The choice of equidistant rotation angles:

$$\theta_{i,j} = j\varphi_i/(1 + m_i), \quad j = 1..m_i, \quad (13)$$

results in $\varphi_{i+1} = \varphi_i/(1 + m_i)$, but other choices might result in simpler implementation, which remains to be investigated in future work. Equidistant angles result in expression for φ_i :

$$\varphi_i = \frac{\pi}{2} \prod_{j=1}^{i-1} (1 + m_j)^{-1}, \quad (14)$$

where the product is one for $i = 1$. Also, the algorithm is fully specified by the sequence of angle divisions m_i . Angle of rotation can be calculated from σ_i and m_i as:

$$\theta = \sum_{i=1}^N \theta_{i,\sigma_i} = \frac{\pi}{2} \sum_{i=0}^N \sigma_i \prod_{j=1}^i (1 + m_j)^{-1} \quad (15)$$

Knowing the required number of $\pi/2$ rotations to put the vector in the fourth quadrant and running (9)–(11) for predefined number of iterations, the sequence σ_i , or equivalently angle θ , which zeroes out one element of matrix **A** is determined. The same rotations need to be applied to other elements of rows i and j , as in (1), to complete the Givens rotation.

The core idea of our algorithm relies on efficient calculation of multi-angle rotations:

$$\mathbf{ROT}(\theta_{i,j}) \begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} x_i \cos(\theta_{i,j}) - y_i \sin(\theta_{i,j}) \\ y_i \cos(\theta_{i,j}) + x_i \sin(\theta_{i,j}) \end{bmatrix} \quad (16)$$

At first glance of (16), it seems that the transformation is too complex for efficient implementation, since it contains multiplications and trigonometric functions and there are $2m_i$ different terms for each iteration. Efficient implementation can be achieved by noting that angle divisions m_i are fixed during the design phase, so $\cos(\theta_{i,j})$ and $\sin(\theta_{i,j})$ are constants. Furthermore, the number of multiplications in the first iteration is halved because $\pi/2 - \theta_{1,j} = \theta_{1,m_1+1-j}$ and consequently $\sin(\theta_{1,j}) = \cos(\theta_{1,m_1+1-j})$. Therefore, the problem of implementing (16) is reduced to the problem of efficient implementation of parallel multiplierless multiple constant multiplication (MMCM).

Efficient implementation of parallel MMCM has been extensively studied in [3,4,14] for the purpose of multiplierless implementation of digital FIR filters. The authors of aforementioned papers have made the Multiplier Block Tool (MBT) available online [13] and as a source code. MBT's input arguments are multiplier constants and the maximum tree depth, from which it generates report about the number of used adders, tree depth and the C code of a MMCM. The number of adders and tree depth for rotations in the first stage of our algorithm was obtained by running MBT, and the results are summarized in Table 1. Results are shown in the form a/b (c/d), where

Table 1 Number of adders and tree depth of $a \sin[k\pi/(2(m_1 + 1))]$, $k = 1, \dots, m_1$ constant multiplier

m_1	8 Bits	12 Bits	16 Bits	20 Bits
1	2/2 (2/2)	3/3 (3/3)	4/3 (4/4)	6/3 (5/4)
2	2/2 (2/2)	4/3 (4/3)	5/3 (5/3)	5/3 (5/4)
3	4/2 (4/2)	6/3 (6/3)	8/3 (8/3)	11/3 (9/5)
4	6/2 (5/3)	7/3 (7/4)	11/3 (8/5)	14/3 (10/6)
5	6/2 (6/2)	8/3 (8/3)	11/3 (10/5)	14/3 (11/5)
6	7/2 (7/3)	9/3 (9/3)	15/3 (13/7)	16/4 (14/6)
7	7/2 (6/5)	11/3 (10/4)	17/3 (15/6)	20/4 (19/7)
8	8/2 (7/3)	11/3 (10/5)	17/3 (15/6)	24/3 (17/7)
9	10/2 (9/3)	14/3 (11/7)	20/3 (15/6)	25/3 (21/6)
10	11/2 (9/3)	17/3 (14/7)	21/3 (18/6)	27/4 (24/7)
11	10/2 (9/4)	16/3 (14/6)	23/3 (19/7)	27/4 (21/7)
12	11/2 (11/2)	18/3 (15/5)	25/3 (21/6)	29/4 (26/12)
13	13/2 (12/3)	18/3 (16/6)	28/3 (24/9)	32/4 (28/9)
14	13/2 (12/4)	18/3 (17/5)	30/3 (22/7)	33/4 (27/10)
15	13/2 (13/2)	21/3 (18/7)	33/3 (24/9)	34/4 (32/7)

a is the number of adders for MMCM with minimum tree depth of b , while c is the minimum number of adders resulting in tree depth d .

For example, first iteration of our algorithm can be implemented with 17 adders and a tree depth of three for 16 bits and $m_1 = 8$. Graph of the full-precision 16-bit, $m_1 = 8$ MMCM generated by MBT is shown in Fig. 2. MBT generates full-precision MMCMs, so the product of two n bit signed numbers has $2n - 1$ bits. This means that adders should also have $2n - 1$ bits. To reduce the complexity and area and increase the operating frequency of adders, we have reduced the precision of the multiplier so that all adders and outputs are in the same fixed point format as input. Reduced-precision MMCM is shown in Fig. 3, with code given in Fig. 4. Round-off errors are minimized by introducing LSB guard bits, which will be discussed later.

Finding the sequence of angle divisions m_i resulting in minimum number of adders can be performed by an exhaustive search for bit width of moderate lengths, while some kind of heuristics should be used for larger number of bits. Exhaustive search algorithm is given in Algorithm 2. The optimum sequences of angle divisions m_i for 16 bit input data and $m_1 = 3, \dots, 15$ have been determined by an exhaustive search, and the results are shown in Table 2. Minimum of 55 adders is achieved for $m_1 = 4$, while the minimum of number of adders and total depth is achieved for $m_1 = 8$. We have selected the sequence of angle divisions $m_i = (8, 4, 2, 2, 1, 1, 1, 1, 1)$ for implementation because it has minimum depth and is second best in terms of number of adders.

3 Software Implementation

The MACM-QR algorithm for 4×4 matrices was first implemented in software to quickly assess the effect of round-off errors and design trade-offs. The results of

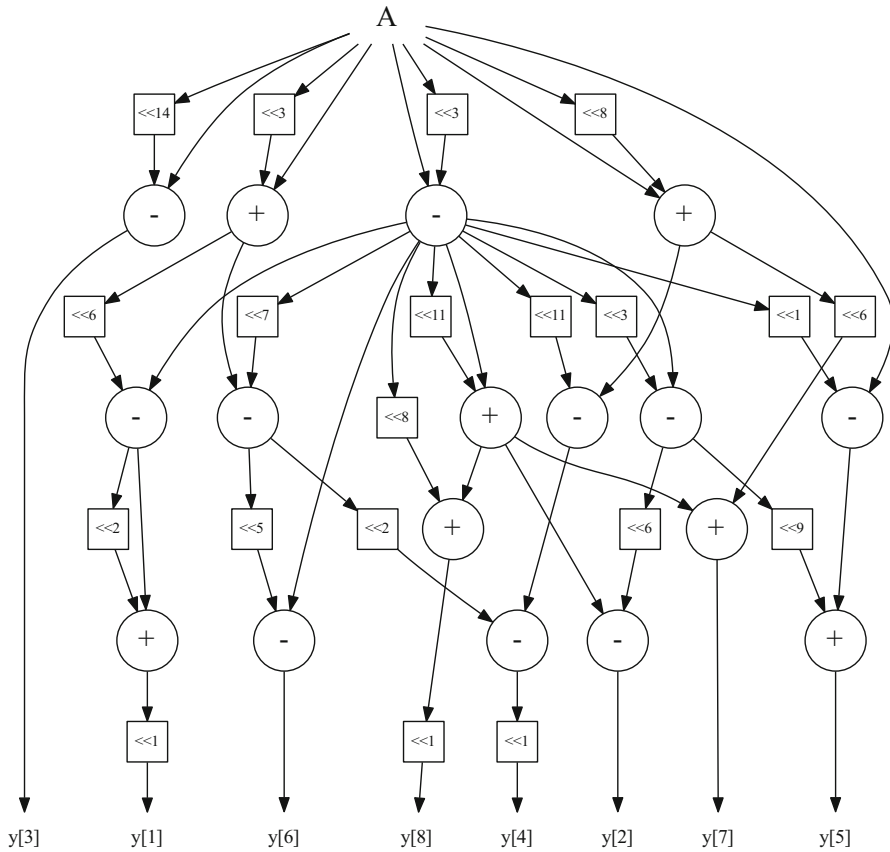


Fig. 2 Full-precision first-stage constant multiplier $y[k] = A \sin(k\pi/18)$, $k = 1 \dots 8$, generated by MBT [13]

software and hardware implementations should be identical to draw valid conclusions. AC Datatypes library from Mentor Graphics Corporation [1] was used because it implements arbitrary length bit-accurate fixed point data type. The library provides type templates and operator overloading with minimum overhead, which makes it ideal candidate for modeling of fixed point arithmetic hardware.

Generation of C++ source code from sequence of angle divisions, bitwidth and number of LSB and MSB guard bits was automated by a Python script, which eliminates the possibility of human error and speeds up the design space exploration. Extension of input data with MSB guard bits is necessary because the result of rotation can be $\sqrt{2}$ times larger than input data. For the case of 4×4 matrices, it is sufficient to add two MSB guard bits. Python script invokes MBT to generate the full-precision MMCMs for each m_i , and reduces its precision to requested number of LSB guard bits. Generation of test matrices, running the tests and processing of test results were also automated by Python scripts.

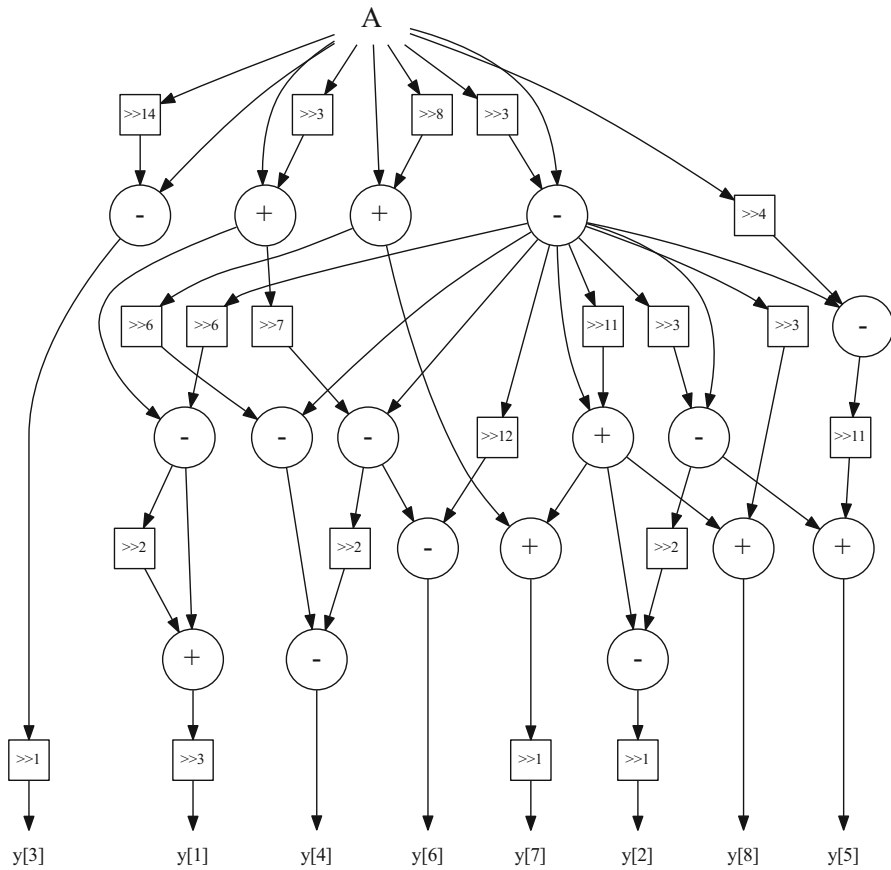


Fig. 3 Reduced-precision first-stage constant multiplier $y[k] = A \sin(k\pi/18)$, $k = 1 \dots 8$, with result in the same fixed point number format as input

Matrix condition number, defined as [6]:

$$\text{cond}(\mathbf{A}) = \frac{\max(s_i)}{\min(s_i)}, \quad (17)$$

where s_i are singular values of $\mathbf{A}$, is a measure of numerical stability. Ill-defined matrices have large condition numbers and are very sensitive to round-off errors. To test the numerical stability of the MACM-QR algorithm, random 4×4 test matrices with condition numbers in the range $[1, 46,000]$, with 3 points per decade and 1000 matrices per condition number, for a total of 15000 test matrices were generated.

Condition number of a random matrix $\mathbf{A}$ was adjusted to the desired value by first performing the singular value decomposition (SVD) $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$. Diagonal entries s_i of matrix $\mathbf{6}$ are singular values of $\mathbf{A}$. Condition number is adjusted by scaling singular values s_i so that $\max(\hat{s}_i)/\min(\hat{s}_i)$ is equal to the desired condition number, and calculating the matrix $\hat{\mathbf{A}} = \mathbf{U}\hat{\mathbf{\Sigma}}\mathbf{V}^*$, where $\hat{\mathbf{\Sigma}}$ contains the scaled singular values

Fig. 4 Code for reduced-precision first-stage constant multiplier
 $y[k] = A \sin(k\pi/18)$,
 $k = 1 \dots 8$, with result in the same fixed point number format as input, shown in Fig. 3

```

t0 = A - A >> 3
t1 = t0 - A >> 4
t2 = A >> 8 + A
t3 = A >> 3 + A
t4 = t0 >> 11 + t0
t5 = t0 - t0 >> 3
t6 = t0 - t3 >> 7
t7 = t3 - t0 >> 6
t8 = t0 - t2 >> 6
y1 = (t7 >> 2 + t7) >> 3
y2 = (t4 - t5 >> 2) >> 1
y3 = (A - A >> 14) >> 1
y4 = t8 - t6 >> 2
y5 = t1 >> 11 + t5
y6 = t6 - t0 >> 12
y7 = (t4 + t2) >> 1
y8 = t4 + t0 >> 3

```

Algorithm 2 Construction of MACM algorithm multipliers

Require: Number of bits N , sequence of angle divisions $m_i, i = 1..k$

```

adders ← 0, depth ← 0
 $\varphi_1 \leftarrow \pi/2$ 
for  $i = 1..k$  do
    constants ← {}
    for  $j = 1..m_i$  do
         $\theta_{i,j} \leftarrow j\varphi_i/(1 + m_i)$ 
        if  $i = 1$  then
            constants ← constants  $\cup$  {int( $2^N \sin(\theta_{i,j})$ )} {First stage has only half of outputs}
        else
            constants ← constants  $\cup$  {int( $2^N \sin(\theta_{i,j})$ )}
            constants ← constants  $\cup$  {int( $2^N \cos(\theta_{i,j})$ )}
        end if
    end for
     $n_{\text{add}}, n_{\text{depth}} \leftarrow \text{MBT}(\text{constants})$ 
    adders ← adders +  $n_{\text{add}}$ , depth ← depth +  $n_{\text{depth}}$ 
     $\varphi_{i+1} \leftarrow \varphi_i/(1 + m_i)$ 
end for
return adders, adders + depth

```

$\hat{s}_i$. Generated test matrices $\hat{\mathbf{A}}$ were written to a file and used in all tests, to ensure consistency of test results.

The first test was run to determine the distribution of errors for different number of LSB guard bits. Test programs were generated with zero to three guard LSBs, and test matrices $\mathbf{A}$ were decomposed with them. Assuming that the error has Gaussian distribution, element-wise error probability density function (PDF) of $\mathbf{A} - \mathbf{QR}$ was calculated and shown in Fig. 5, with number of LSB guard bits as parameter. Errors

Table 2 Optimum angle division sequences for 16 bit data

m_1	ADD	ADD + Depth	Angle division sequence m_i
3	58	77	(3, 2, 2, 4, 2, 2, 1, 1, 1)
4	55	72	(4, 2, 2, 2, 2, 1, 1, 1, 1, 1)
5	58	75	(5, 2, 2, 2, 4, 1, 1, 1, 1)
6	63	83	(6, 2, 2, 1, 2, 2, 2, 1, 1)
7	62	80	(7, 2, 2, 4, 2, 2, 1, 1)
8	57	71	(8, 4, 2, 2, 1, 1, 1, 1, 1)
9	61	75	(9, 4, 3, 1, 1, 1, 1, 1, 1, 1)
10	59	76	(10, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1)
11	63	78	(11, 2, 4, 2, 2, 1, 1, 1)
12	62	80	(12, 1, 1, 2, 4, 1, 1, 1, 1, 1)
13	67	84	(13, 2, 2, 2, 2, 2, 1, 1)
14	62	75	(14, 2, 2, 2, 1, 1, 1, 1, 1)
15	73	86	(15, 4, 4, 1, 1, 1, 1, 1, 1)

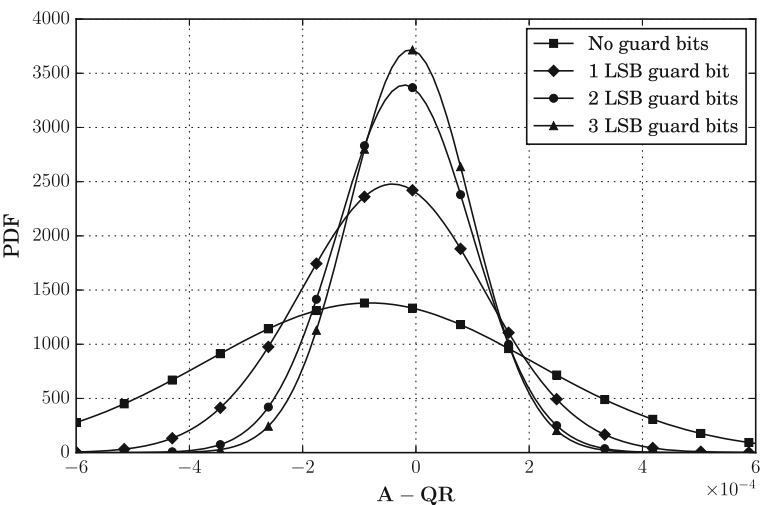


Fig. 5 Probability density function of element-wise $\mathbf{A} - \mathbf{QR}$ error with the number of LSB guard bits as parameter

of QR decompositions with zero and one LSB guard bits are biased, since their mean value is not zero. Mean error of QR decomposition with two LSB guard bits is close to zero, and increasing the number of guard bits to three results in marginal improvement. This test suggests that two LSB guard bits are sufficient.

The second test was run to determine the Frobenius norm [6, pp. 55] of $\mathbf{A} - \mathbf{QR}$ with number of LSB guard bits as parameter, and the results are shown in Fig. 6. The Frobenius norm $\|\mathbf{A} - \mathbf{QR}\|_F$ was calculated for each matrix, and maximum and minimum values within a bin size of 64 matrices were plotted. As in the first test, increasing

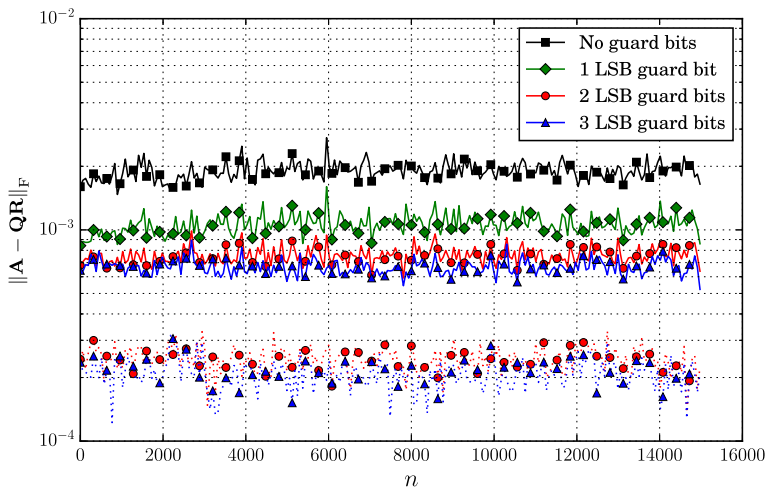


Fig. 6 Frobenius norm $\|\mathbf{A} - \mathbf{QR}\|_F$ with the number of LSB guard bits as parameter. Solid and dashed lines represent the maximum and minimum norms in bins of 64 matrices, respectively

the number of LSB guard bits above two yields in marginal improvements. Although condition number of test matrices is in wide range, Frobenius norm maximum and minimum values are almost constant.

To verify that the maximum error is well behaved, matrices were divided into sets with the same condition number. Maximum norm $\|\mathbf{A} - \mathbf{QR}\|_{\max}$ was calculated for each matrix in a set, and the maximum value for each set of matrices is given in Table 3 for different number of LSB guard bits. Maximum error is greatest when there are no LSB guard bits, and decreases as more guard bits are added, as expected. Again, going from two to three guard bits results in marginal improvement.

Authors of CORDIC-based implementation in [12] suggest that minimum error is achieved for reduced number of iterations, which is equivalent to determining the rotation angle with reduced precision. Their interpretation is that the accumulated rounding error starts to dominate after a certain number of CORDIC iterations. We have not observed such an effect in our algorithm with two LSB guard bits, and the error always decreases as more iterations are run.

4 Hardware Implementation

Hardware implementation of MACM-QR algorithm was automated with Python scripts which generate the Verilog description from specified number of bits, angle divisions m_i and pipelining plan. One stage of MACM-QR, shown in Fig. 7, works exactly as in the software implementation and consists of two MMCMs, adders, thermometer to binary (priority) encoder and multiplexers. Selection of rotated vector closest to x axis is accomplished by connecting the sign bits of $Y[i]$ to inputs of thermometer to binary encoder, which counts the number of consecutive ones. Example given in Fig. 1 would have 00111 as an input to thermometer to binary encoder because

Table 3 Error matrix max norm

cond(A)	max ($\ \mathbf{A} - \mathbf{QR}\ _{\max}$)			
	0 LSB	1 LSB	2 LSB	3 LSB
1	9.46E-04	5.80E-04	4.12E-04	3.85E-04
2.2	1.13E-03	7.32E-04	5.11E-04	3.89E-04
4.6	1.10E-03	6.87E-04	4.88E-04	4.20E-04
10	1.37E-03	7.48E-04	4.58E-04	3.89E-04
22	1.22E-03	7.32E-04	4.50E-04	4.01E-04
46	1.19E-03	7.02E-04	4.58E-04	3.74E-04
100	1.28E-03	6.41E-04	4.88E-04	4.16E-04
220	1.22E-03	6.71E-04	4.50E-04	3.78E-04
460	1.31E-03	7.48E-04	4.73E-04	4.01E-04
1000	1.31E-03	7.63E-04	4.88E-04	4.43E-04
2200	1.40E-03	8.09E-04	4.96E-04	4.04E-04
4600	1.19E-03	7.32E-04	5.04E-04	4.08E-04
10000	1.19E-03	7.32E-04	5.04E-04	4.08E-04
22000	1.25E-03	7.32E-04	5.19E-04	3.78E-04
46000	1.34E-03	7.78E-04	5.19E-04	4.23E-04

first three vectors have $Y[i] < 0$, and the output would be 011 (decimal 3), selecting the vector $Y[3]$ as output.

MMCMs in the first stage have only half of the outputs because $\theta_{1,j} = j\pi/(2(m_1 + 1))$, and consequently $\sin \theta_{1,j} = \cos \theta_{1,m_1+1-j}$. Control bit MODE selects whether the stage operates in rotation (MODE = 0) or vectoring mode (MODE = 1). In vectoring mode, the input vector is rotated toward the positive x axis and the chosen angle σ_i of rotation is saved in register REG. In rotation mode, the saved angle is used to rotate the input vector (Fig. 8).

Generated MACM-QR core was synthesized for Xilinx Virtex XC6VLX75T-3 with ISE WebPack software. Pipelining is an effective technique to increase the operating frequency at a cost of increased latency. Achieving higher operating frequency at a cost of increased latency might be acceptable when the data throughput is a priority. In other applications, lower operating frequency might be acceptable when low latency is important. Therefore, latency–operating frequency trade-off should be explored.

Complex structure of MMCMs (Fig. 3) and MACM-QR stage (Fig. 7) makes the identification of optimum pipeline register placement difficult. Most synthesis tools support register balancing (retiming), so it is sufficient to place a chosen number of registers at the output of each stage, and the synthesis tool will move them to appropriate position to maximize the operating frequency. For a specified latency of k clocks, a total of k registers should be distributed across N stages. This is equivalent to the problem of putting k indistinguishable balls in N distinguishable boxes with no restriction on the number of balls in a box, with a number of possible choices $\binom{N+k-1}{k}$.

The number of possible pipeline register placements for latency of k clocks can be very large, so it is necessary to find a way to reduce it. For each stage, there are a

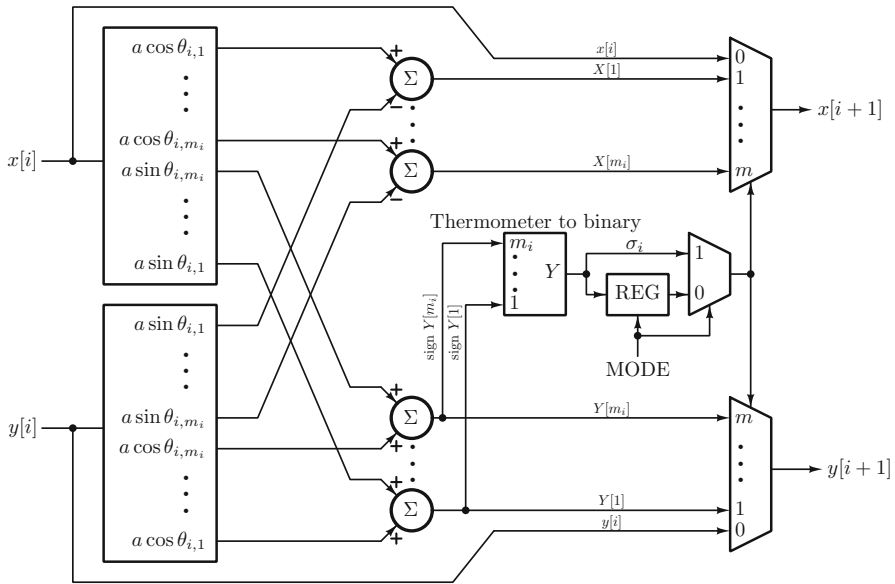


Fig. 7 Hardware implementation of MACM-QR stage

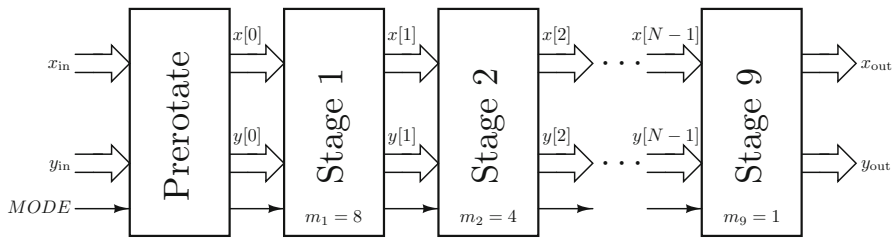


Fig. 8 Hardware implementation of MACM-QR pipeline

maximum number of pipeline registers which will increase the operating frequency, after which the operating frequency remains the same. Maximum number of pipeline registers depends on the FPGA technology and architecture, and the complexity of the stage itself, so it is difficult to estimate from Verilog code, but can be determined by a simple experiment. To determine the maximum number of pipeline registers for a given stage, a series of synthesis runs are performed with a variable number of pipeline registers from zero up to a number when the operating frequency saturates, while the other stages have a large number of pipeline registers and are already operating at a maximum frequency, e.g., 10. The result of such frequency sensitivity experiment is summarized in Table 4.

Examining the results from Table 4 reveals that using more than three pipeline registers for stage 1 does not yield any improvement. Synthesis results also show that adding pipeline registers does not increase the operating frequency of some stages, e.g., stages 6 to 9, suggesting that they could be merged. Using this information, the number of possible pipeline register placements can be significantly reduced, so that in most

Table 4 Maximum operating frequency versus number of pipeline registers per stage

Stage	max f (MHz)			
	No reg	1 Reg	2 Reg	3 Reg
1	126.8	208.7	247.6	269.8
2	111.2	195.1	269.8	269.8
3	137.3	215.8	269.8	269.8
4	185.3	269.8	269.8	269.8
5	231.5	269.8	269.8	269.8
6	269.8	269.8	269.8	269.8
7	269.8	269.8	269.8	269.8
8	269.8	269.8	269.8	269.8
9	269.8	269.8	269.8	269.8

Table 5 Top three maximum operating frequencies versus latency

Latency	f (MHz)		
	1st	2nd	3rd
2	77.2	66.9	66.4
3	113.9	109.5	100.0
4	132.8	132.8	122.8
5	162.8	161.0	151.6
6	188.3	187.8	180.5

cases all of them can be synthesized to determine the maximum operating frequency for specified latency. Top three maximum operating frequencies as a function of latency, i.e., total number of pipeline registers, are given in Table 5. Implementation of MACM-QR core with highest operating frequency (6 clocks latency) used 766 slice registers and 3108 slice LUTs, which is 6% of XC6VLX75T device.

The presented algorithm can be used for parallel QR factorization as was done in [12] at the expense of additional resources. However, low latency enables the use of a single MACM-QR core in a sequential manner as well.

5 Conclusion

In this paper, we have presented a new algorithm for implementing Givens rotation for use in matrix QR decomposition. The core idea of the algorithm relies on efficient implementation of multi-angle micro-rotations by using multiplierless multi-constant multipliers. Bit-exact algorithm was implemented in software by using a fixed point library to test its numerical properties and explore the optimum angle division sequences. Algorithm has been used for QR decomposition of 4×4 matrices and is numerically stable, since increasing the number of bits always results in increased precision, unlike some CORDIC implementations which exhibit a minimum of error for reduced precision due to round-off error accumulation. Hardware implementation

of 16-bit precision MACM-QR core on Xilinx Virtex 6 device uses 766 slice registers and 3108 slice LUTs and can operate at a frequency of 188.3 MHz with 6 cycles of latency.

References

1. AC Datatypes v3.7 (2016). <https://www.mentor.com/hls-lp/downloads/ac-datatypes>. Accessed June 2016
2. P.L. Chiu, L.Z. Huang, L.W. Chai, Y.H. Huang, Interpolation-based QR decomposition and channel estimation processor for MIMO-OFDM system. *IEEE Trans. Circuits Syst. I Regul. Pap.* **58**(5), 1129–1141 (2011). <https://doi.org/10.1109/TCSI.2010.2092090>
3. A.G. Dempster, M.D. Macleod, Constant integer multiplication using minimum adders. *IEE Proc. Circuits Devices Syst.* **141**(5), 407–413 (1994)
4. A.G. Dempster, M.D. Macleod, Use of minimum-adder multiplier blocks in FIR digital filters. *IEEE Trans. Circuits Syst. II Analog Digit. Signal Process.* **42**(9), 569–577 (1995)
5. M.D. Ercegovic, T. Lang, *Digital Arithmetic* (Elsevier, Amsterdam, 2004)
6. G.H. Golub, C.F. Van Loan, *Matrix Computations*, vol. 3, 3rd edn. (JHU Press, Baltimore, 2003)
7. X. Hu, R. Harber, S. Bass, Expanding the range of convergence of the CORDIC algorithm. *IEEE Trans. Comput.* **40**(1), 13–21 (1991). <https://doi.org/10.1109/12.67316>
8. Z.Y. Huang, P.Y. Tsai, Efficient implementation of QR decomposition for gigabit MIMO-OFDM systems. *IEEE Trans. Circuits Syst. I Regul. Pap.* **58**(10), 2531–2542 (2011). <https://doi.org/10.1109/TCSI.2011.2123770>
9. J.S. Lin, Y.T. Hwang, S.H. Fang, P.H. Chu, M.D. Shieh, Low-complexity high-throughput QR decomposition design for MIMO systems. *IEEE Trans. Very Large Scale Integr. Syst.* **23**(10), 2342–2346 (2015). <https://doi.org/10.1109/TVLSI.2014.2361906>
10. L. Ma, K. Dickson, J. McAllister, J. McCanny, QR decomposition-based matrix inversion for high performance embedded MIMO receivers. *IEEE Trans. Signal Process.* **59**(4), 1858–1867 (2011). <https://doi.org/10.1109/TSP.2011.2105485>
11. P. Meher, J. Valls, J. Tso-Bing, K. Sridharan, K. Maharatna, 50 Years of CORDIC: algorithms, architectures, and applications. *IEEE Trans. Circuits Syst. I Regul. Pap.* **56**(9), 1893–1907 (2009). <https://doi.org/10.1109/TCSI.2009.2025803>
12. S.D. Munoz, J. Hormigo, High-throughput FPGA implementation of QR decomposition. *IEEE Trans. Circuits Syst. II Express Briefs* **62**(9), 861–865 (2015)
13. Spiral: Software/Hardware Generation for DSP Algorithms (2016). <http://spiral.ece.cmu.edu/mcm/gen.html>. Accessed June 2016
14. Y. Voronenko, M. Püschel, Multiplierless multiple constant multiplication. *ACM Trans. Algorithms (TALG)* **3**(2), 11 (2007)
15. C. Zhang, H. Prabhu, Y. Liu, L. Liu, O. Edfors, V. wall, Energy efficient group-sort QRD processor with on-line update for MIMO channel pre-processing. *IEEE Trans. Circuits Syst. I Regul. Pap.* **62**(5), 1220–1229 (2015). <https://doi.org/10.1109/TCSI.2015.2402936>
16. W. Zhao, J. Lin, S.C. Chan, Throughput/area efficient FPGA implementation of QR decomposition for MIMO systems, in *2016 IEEE International Conference on Digital Signal Processing (DSP)* (2016), pp. 522–526. <https://doi.org/10.1109/ICDSP.2016.7868612>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.